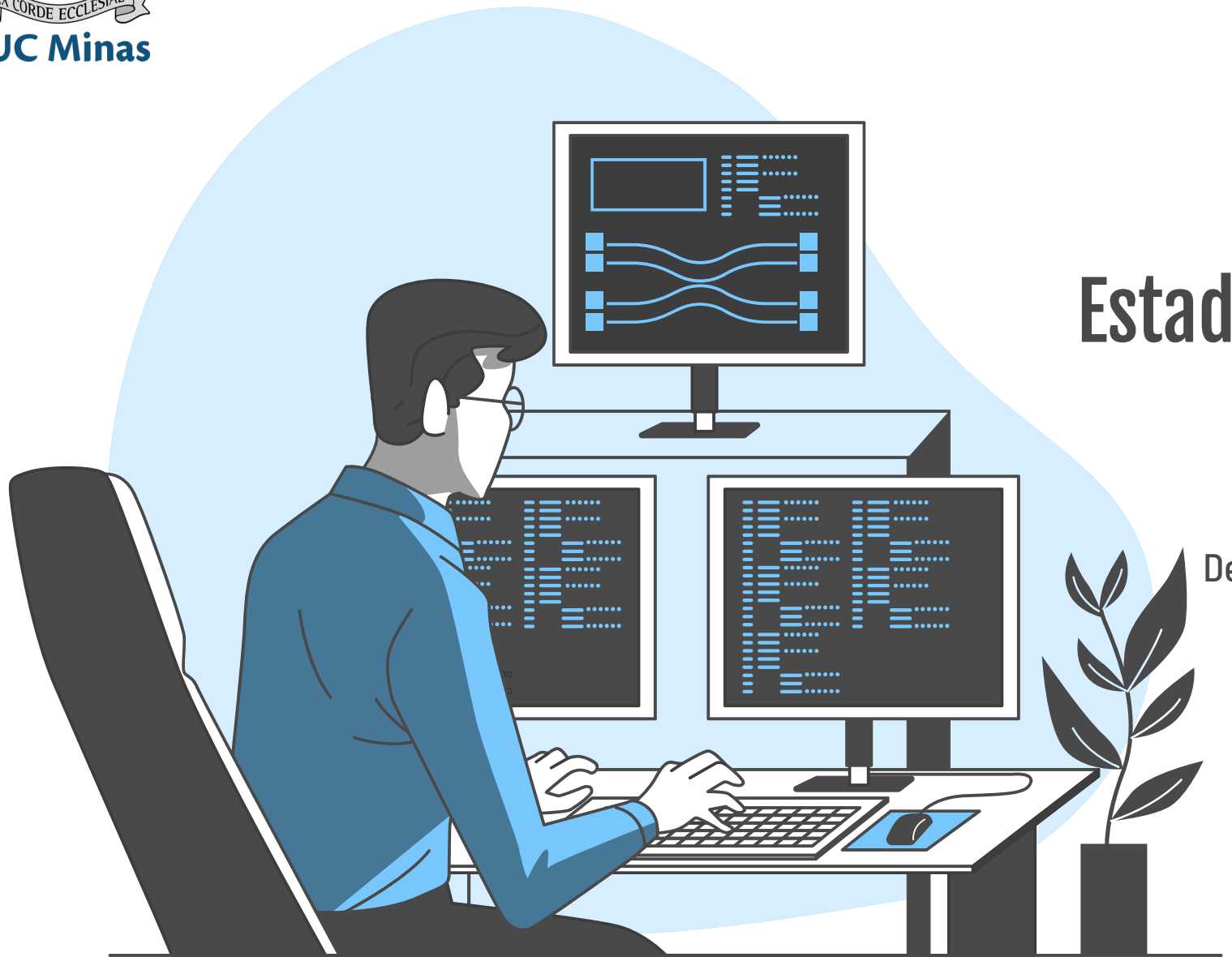
PUC Minas



Aula 05 · Front-end

Estado, eventos e formulários

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender o que é **estado** e por que uma variável comum não atualiza a tela

2

Usar `useState` com números, textos, objetos e listas

3

Tratar **eventos** (`onClick`, `onChange`, `onSubmit`)

4

Criar **formulários controlados**

5

Elevar o estado e passar funções como props

Antes de começar

- Aula 04: componentes e props

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Por que uma variável comum não funciona

02 Atualizando objetos e listas

03 Eventos

04 Formulário controlado

05 Elevando o estado

06 Aprofundando: estado bem organizado

01

Por que uma variável comum não funciona

Por que uma variável comum não funciona

Errado

```
function Contador() {
  let cliques = 0;
  return <button onClick={() => { cliques++; console.log(cliques); }}>{cliques}</button>;
}
// O console mostra 1, 2, 3... mas a tela continua mostrando 0.
```

Por que uma variável comum não funciona

O React só redesenha um componente quando o **estado** dele muda. Uma variável `let` é recriada com `0` a cada renderização e o React nem sabe que ela mudou. Para isso existe o *hook* **`useState`**.

Certo: com useState

```
import { useState } from "react";

function Contador() {
  const [cliques, setCliques] = useState(0);   // [valor atual, função que altera]

  return (
    <button onClick={() => setCliques(cliques + 1)}>
      Cliquei {cliques} vezes
    </button>
  );
}
```

Por que uma variável comum não funciona

- `useState(0)` cria o estado com valor inicial `0` e devolve um par: o valor e a função *setter*.
- Chamar `setCliques(...)` **agenda** uma nova renderização com o novo valor.
- *Hooks* (funções que começam com `use`) só podem ser chamados **no topo** do componente, nunca dentro de `if`, `for` ou funções internas.

01

Por que uma variável comum não funciona

Aula 05 · Front-end

02

Atualizando objetos e listas

Atualizando objetos e listas

O estado deve ser tratado como **imutável**: em vez de alterar o objeto/array, crie um **novo** com o *spread* (`...`).

Imutabilidade

```
const [aluno, setAluno] = useState({ nome: "", curso: "" });
setAluno({ ...aluno, nome: "Ana" });                  // copia e troca um campo

const [lista, setLista] = useState([]);
setLista([...lista, novoItem]);                        // adicionar
setLista(lista.filter((a) => a.id !== id));            // remover
setLista(lista.map((a) => (a.id === id ? { ...a, curso: "ES" } : a)));   // editar

// quando o novo valor depende do anterior, use a forma de função:
setCliques((anterior) => anterior + 1);
```

Atualizando objetos e listas

Erro comum

`lista.push(item); setLista(lista);` não atualiza a tela: o array é o mesmo objeto, então o React acha que nada mudou. Sempre crie um array novo.

03

Eventos

Eventos

Evento	Elemento	Recebe
`onClick`	botões, qualquer elemento	`MouseEvent`
`onChange`	`input`, `select`, `textarea`	`e.target.value`
`onSubmit`	`form`	chame `e.preventDefault()` para não recarregar a página
`onKeyDown`	inputs	`e.key === "Enter"`

Eventos

Passe a **função**, não o resultado dela: `onClick={salvar}` ou `onClick={() => excluir(id)}`. Escrever `onClick={excluir(id)}` executa na hora em que a tela é desenhada.

03

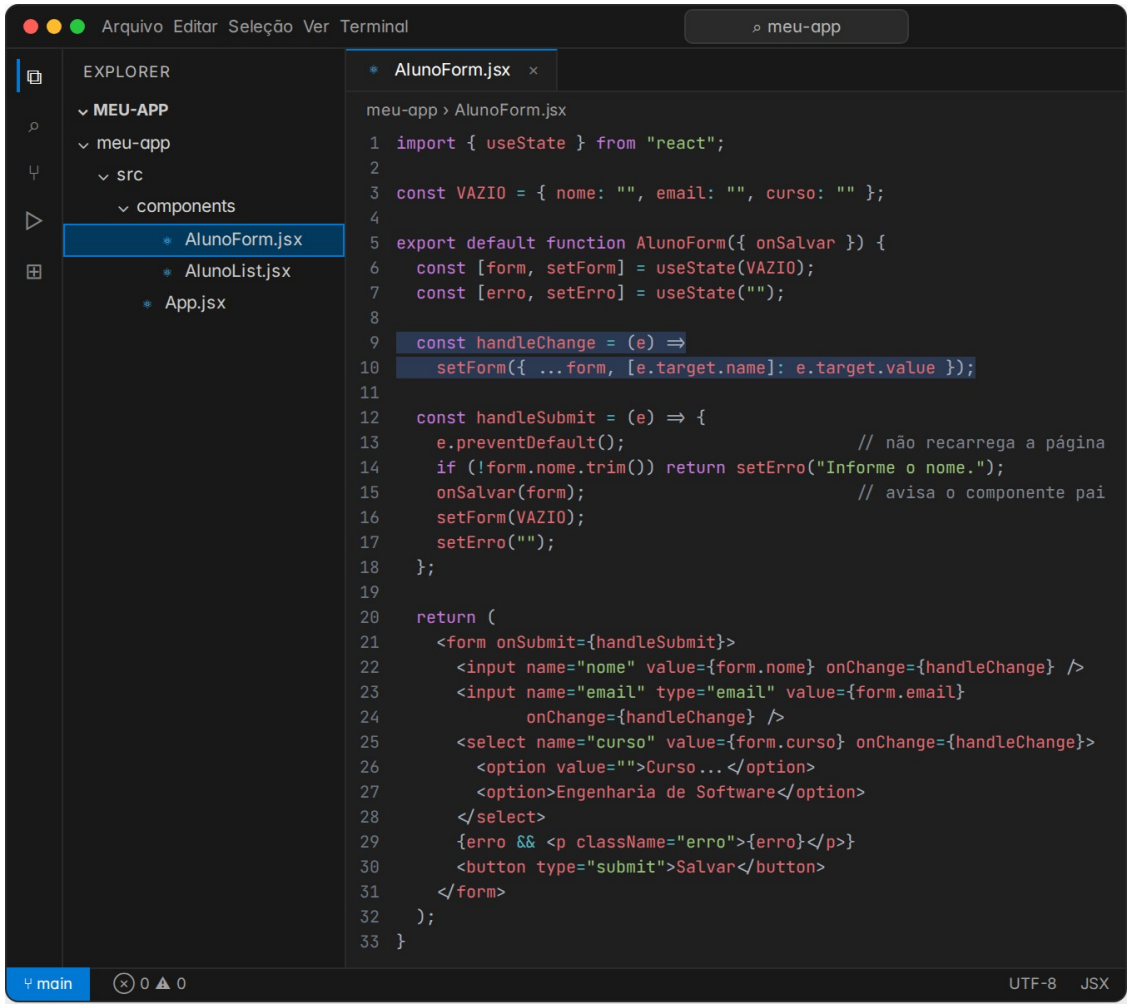
Eventos

Aula 05 · Front-end

04

Formulário controlado

Formulário controlado



O valor de cada campo vive no estado; o campo mostra o estado e o `onChange` o atualiza. Um único `handleChange` atende todos os campos usando o atributo `name`.

Formulário controlado, no mesmo formato do `AlunoForm` do projeto `Exemplo_Integracao`.

05

Elevando o estado

Elevando o estado

Quando **dois componentes** precisam do mesmo dado (o formulário adiciona, a lista mostra), o estado fica no **pai comum**. O pai passa o dado para um filho por props e passa uma **função** para o outro filho avisar mudanças.

App.jsx (dono do estado)

```jsx
export default function App() {
  const [alunos, setAlunos] = useState([]);

  const adicionar = (dados) =>
    setAlunos([...alunos,
      { ...dados, id: crypto.randomUUID() }]);

  const excluir = (id) =>
    setAlunos(alunos.filter((a) => a.id !== id));

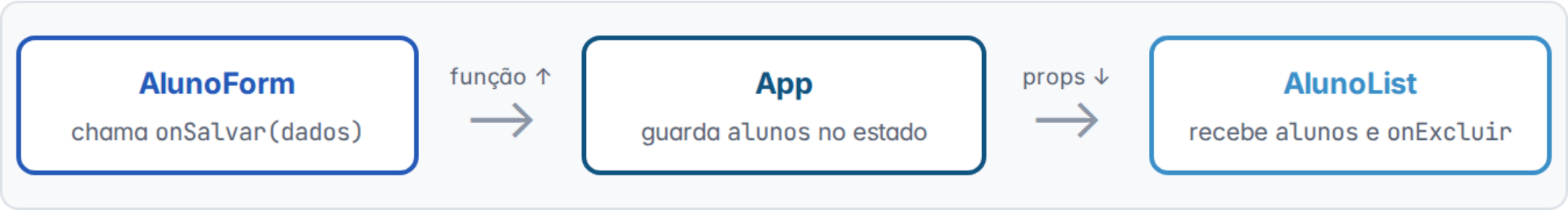
  return (
    <>
      <AlunoForm onSalvar={adicionar} />
      <AlunoList alunos={alunos}
                 onExcluir={excluir} />
    </>
  );
}
```

AlunoList.jsx

```jsx
export default function AlunoList({
  alunos, onExcluir }) {
  if (alunos.length === 0)
    return <p>Nenhum aluno cadastrado.</p>;

  return (
    <ul>
      {alunos.map((a) => (
        <li key={a.id}>
          {a.nome} - {a.curso}
          <button onClick={() =>
            onExcluir(a.id)}>Excluir</button>
        </li>
      ))}
    </ul>
  );
}
```

Elevando o estado



Dados descem por props; eventos sobem por funções.

Elevando o estado

Dica

Instale a extensão **React Developer Tools** no navegador. A aba *Components* mostra a árvore de componentes com as props e o estado de cada um, e permite editar valores ao vivo.

06

Aprofundando: estado bem organizado

Onde colocar cada estado

Só um componente usa

Fica nele (`useState` local).

Irmãos usam

Sobe para o pai comum.

O app todo usa

Context (usuário logado, tema) — aula 10.

Pode ser calculado

Não é estado. Calcule na renderização.

Aprofundando: estado bem organizado

Estado derivado

```
// ruim: estado duplicado que pode ficar inconsistente
const [alunos, setAlunos] = useState([]);
const [total, setTotal] = useState(0);        // precisa lembrar de atualizar

// bom: derive na hora
const [alunos, setAlunos] = useState([]);
const total = alunos.length;
const ativos = alunos.filter((a) => a.ativo);
```

Atualizações em lote

O React agrupa várias chamadas de `set` feitas no mesmo evento e redesenha **uma vez só**. Por isso, ler o estado logo após o `set` ainda mostra o valor antigo: a atualização acontece na próxima renderização.

Forma de função quando o novo valor depende do anterior

```
function handleClick() {
  setCliques(cliques + 1);
  setCliques(cliques + 1);
  console.log(cliques);   // ainda mostra o valor antigo
  // resultado: +1, não +2 (as duas usaram o mesmo valor)
}

function handleClickCerto() {
  setCliques((c) => c + 1);
  setCliques((c) => c + 1);   // +2: cada uma usa o valor anterior
}
```

Formulários com vários tipos de campo

Campo	Atributo controlado	Valor lido no onChange
``	`value`	`e.target.value`
``	`value`	`Number(e.target.value)`
``	`checked`	`e.target.checked`
``	`value`	`e.target.value`
``	`value`	`e.target.value`

Controlado x não controlado

Controlado (`value` + `onChange`)

- O React sempre sabe o valor
- Permite validar a cada tecla
- Padrão dos projetos da disciplina

vs

Não controlado (`useRef`)

- O valor vive no próprio input
- Lido só no submit
- Útil para inputs de arquivo e formulários enormes

Aprofundando: estado bem organizado

Validação em tempo real

```
const emailValido = /^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(form.email);
const podeSalvar = form.nome.trim() !== "" && emailValido;

<input name="email" value={form.email} onChange={handleChange}
       aria-invalid={!emailValido} />
{!emailValido && form.email && <small className="erro">E-mail inválido</small>}
<button disabled={!podeSalvar}>Salvar</button>
```

Aprofundando: estado bem organizado

Dica

Mesmo validando no front, **o back-end valida de novo** (`@Valid` no Spring). A validação do front é para a experiência do usuário; a do back é para a segurança dos dados.

07

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que `let cliques = 0; cliques++` não atualiza a tela?

A Falta CSS

B Só mudanças de estado fazem o React redesenhar

C let é proibido

D Falta o key

Quiz · questão 1 de 5

Resposta

Por que `let cliques = 0; cliques++` não atualiza a tela?

A. Falta CSS

✓ Só mudanças de estado fazem o React redesenhar

C. let é proibido

D. Falta o key

Por quê: Use useState.

Pergunta

Como adicionar um item a um array de estado?

A lista.push(x); setLista(lista)

B setLista([...lista, x])

C lista[lista.length] = x

D setLista(lista + x)

Resposta

Como adicionar um item a um array de estado?

A lista.push(x); setLista(lista)

✓ setLista([...lista, x])

C lista[lista.length] = x

D setLista(lista + x)

Por quê: Crie sempre um array novo.

Pergunta

Qual a forma correta de passar um evento com parâmetro?

A onClick={excluir(id)}

B onClick={() => excluir(id)}

C onClick="excluir(id)"

D onclick={excluir}

Resposta

Qual a forma correta de passar um evento com parâmetro?

A onClick={excluir(id)}

✓ onClick={() => excluir(id)}

C onClick="excluir(id)"

D onclick={excluir}

Por quê: A primeira executa na renderização.

Pergunta

Em um formulário controlado, o `value` do input vem de:

A O DOM

B O estado do componente

C O localStorage

D O CSS

Resposta

Em um formulário controlado, o `value` do input vem de:

A O DOM

✓ O estado do componente

C O localStorage

D O CSS

Por quê: O onChange atualiza esse estado.

Pergunta

Quando dois componentes irmãos precisam do mesmo dado, onde fica o estado?

A Em cada um

B No pai comum

C No index.html

D Em variável global

Resposta

Quando dois componentes irmãos precisam do mesmo dado, onde fica o estado?

A Em cada um

✓ No pai comum

C No index.html

D Em variável global

Por quê: É o “elevar o estado”.

Fechamento

Resumo da aula

- ✓ Só **estado** (`useState`) faz o componente redesenhar; variáveis comuns não.
- ✓ Estado é imutável: crie objetos/arrays novos com `...`, `filter`, `map`.
- ✓ Eventos recebem **funções**; no `onSubmit` use `e.preventDefault()`.
- ✓ Formulário controlado: `value` vem do estado, `onChange` atualiza.
- ✓ Estado compartilhado fica no **pai**; filhos recebem dados e funções por props.

Exercícios

1. Faça um contador com botões +1, -1 e zerar, sem permitir valores negativos.

2. Crie uma lista de tarefas com adicionar, marcar como concluída e excluir.

3. Adicione validação de e-mail ao `AlunoForm` e desabilite o botão enquanto o formulário for inválido.

4. Implemente a edição: clicar em um aluno preenche o formulário, e salvar substitui o item.

Para ir além

- react.dev/learn/state-a-components-memory
- react.dev/learn/sharing-state-between-components

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 05?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web